# CPSC-354 Report

Angelina House
Chapman University

October 26, 2025

**Abstract**

This report documents my learning and progress throughout the Fall 2025 semester in CPSC-354: Programming Languages.

## Contents

## 1 Introduction

This report chronicles my work in CPSC-354: Programming Languages during Fall 2025. Each weekly subsection contains restated homework prompts, fully typeset solutions, and a brief reflection or question that ties the technical material to broader themes.

## 2 Week by Week

### 2.1 Week 1

#### Homework

Discuss the MU puzzle from *Gödel, Escher, Bach* (Chapter 1; see course PDF pp. 41–42). Does the string `MU` have a derivation in the MIU formal system starting from `MI`? Why or why not?

**The MU (MIU) System Recap**

We begin with the single axiom string `MI`. The system has four production rules:

1. If a string ends in `I`, append a `U` at the end.
2. If the string has the form `Mx`, duplicate everything after `M`: `Mx` $\rightarrow$ `Mxx`.
3. Replace any occurrence of `III` by `U`.
4. Delete any occurrence of `UU`.

From `MI`, we can derive many strings by applying the rules:

`MI`(2)`MII`(2)`MIIII`(3)`MUI`.

This illustrates how the rules expand and compress substrings, but it remains unclear how to end with exactly `MU`.

Let $I(s)$ be the number of `I`'s in a string $s$. Initially, $I(\texttt{MI}) = 1$, i.e., 1 (mod 3). Examine how each rule affects $I$ modulo 3:

- Rule 1 adds a `U`; $I$ is unchanged.
- Rule 2 doubles the count of `I`'s: $I \mapsto 2I$. Thus if $I \equiv 1 \pmod 3$ then $2I \equiv 2 \pmod 3$.
- Rule 3 subtracts three `I`'s (`III` $\to$ `U`); the residue modulo 3 is unchanged.
- Rule 4 deletes `U`'s only; $I$ is unchanged.

Therefore, starting from $I \equiv 1 \pmod 3$, any derivable string keeps $I \equiv 1$ or $I \equiv 2 \pmod 3$, but never $0 \pmod 3$. The target `MU` has $I(\texttt{MU}) = 0$ ($0 \pmod 3$), so it is *not* derivable. Hence, the MU puzzle has no solution.

The puzzle shows how stepping outside a formal system and using an invariant (here, the count of `I`'s modulo 3) can decisively answer a reachability question that seems open-ended from within the rules.

#### Reading Summary & Question

**Summary.** Hofstadter introduces formal systems via the MIU puzzle and contrasts *inside-the-system* rule manipulation with *meta-level* reasoning using invariants. This frames later topics on meaning, proof, and limitations of formal systems.

**Question.** Are there any situations where the MU puzzle could be true, and how much would we have to change to get to that point?

### 2.2 Week 2

## Homework

Using the course notes on abstract reduction systems (ARS), determine for each of the seven given ARSs whether it is *terminating*, *confluent*, and whether it *has unique normal forms (UN)*. Then, fill the grid of the eight possible (Confluent, Terminating, Has UN) combinations with examples or explain why some combinations are impossible.

#### Quick Definitions

- **Terminating:** there is no infinite chain $x_0 \to x_1 \to x_2 \to \cdots$.
- **Confluent:** if $x \to^* y$ and $x \to^* z$, then $y$ and $z$ are joinable.
- **Has unique normal forms (UN):** every element has a (and in fact the) unique normal form.

#### Seven ARSs: Diagrams & Properties

Below, each ARS is shown as a tiny diagram (compiled with `tikz-cd`) followed by its properties and a one–line justification.

**1)** $A = \varnothing$**.**

(empty set, no nodes or arrows)

**Props:** Terminating ✓, Confluent ✓, UN ✓. *Reason:* All properties hold vacuously.

**2)** $A = \{a\},\ R = \varnothing$**.**

$a$

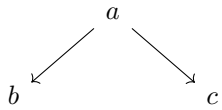
**Props:** Terminating ✓, Confluent ✓, UN ✓. *Reason:* No arrows; $a$ is a (unique) normal form.

**3)** $A = \{a\},\ R = \{(a, a)\}$**.**




**Props:** Terminating ×, Confluent ✓, UN ×. *Reason:* Self–loop gives infinite chain; only one object so forks (if any) are trivially joinable; no normal form exists.

**4)** $A = \{a, b, c\},\ R = \{(a, b), (a, c)\}$**.**



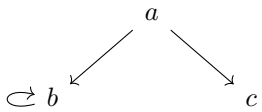

**Props:** Terminating ✓, Confluent ×, UN ×. *Reason:* From $a$ reach distinct NFs $b$ and $c$ that are not joinable.

**5)** $A = \{a, b\},\ R = \{(a, a), (a, b)\}$**.**



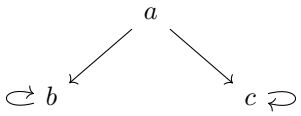

**Props:** Terminating ×, Confluent ✓, UN ✓. *Reason:* There exists an infinite chain via the loop on $a$, but all reductions from $a$ can join at $b$, and every element's normal form is uniquely $b$.

**6)** $A = \{a, b, c\},\ R = \{(a, b), (b, b), (a, c)\}$**.**




**Props:** Terminating ×, Confluent ×, UN ×. *Reason:* Loop at $b$ breaks termination; from $a$ you can reach $b$ (non–NF forever) or $c$ (an NF), so confluence and UN fail.

**7)** $A = \{a, b, c\},\ R = \{(a, b), (b, b), (a, c), (c, c)\}$**.**




**Props:** Terminating ×, Confluent ×, UN ×. *Reason:* Loops at $b$ and $c$; from $a$ diverge to non–joinable loops; no normal forms.

**Summary Table (Seven ARSs)**

| # | Terminating | Confluent | Has UN | Quick reason |
|---|---|---|---|---|
| 1 | ✓ | ✓ | ✓ | Empty set: vacuous |
| 2 | ✓ | ✓ | ✓ | Single node, no edges |
| 3 | × | ✓ | × | Self–loop; no NF |
| 4 | ✓ | × | × | $a \to b$ or $a \to c$ (two NFs) |
| 5 | × | ✓ | ✓ | Loop at $a$ but unique NF $b$ |
| 6 | × | × | × | Loop at $b$; $a \to b$ vs $a \to c$ |
| 7 | × | × | × | Loops at $b, c$; $a \to b$ vs $a \to c$ |

**Eight Combinations Grid**

| Confluent | Terminating | Has UN | Example or reason |
|---|---|---|---|
| T | T | T | Example #2 (or #1) |
| T | T | F | **Impossible:** Confluence+termination ⇒ unique NF |
| T | F | T | Example #5 |
| T | F | F | Example #3 |
| F | T | T | **Impossible:** UN in a terminating system ⇒ confluent |
| F | T | F | Example #4 |
| F | F | T | **Impossible:** UN (for all elements) ⇒ confluent |
| F | F | F | Example #6 or #7 |

**Reflection & Question**

**Question.** What is a real life example of something that has two possible end states like in ARS #4 where $a$ can go to either $b$ or $c$?

## 2.3 Week 3

## Homework

**Exercise 5 — Analysis**

**Rules:**

$$\mathtt{ab} \to \mathtt{ba}, \quad \mathtt{ba} \to \mathtt{ab}, \quad \mathtt{aa} \to \varepsilon.$$

**Sample reductions.**

$$\mathtt{abba} \;\Rightarrow\; \mathtt{baba\,ba} \to \mathtt{ab\,bbaa\,aa} \to \varepsilon\,\mathtt{bb}.$$

$$\mathtt{bababa} \;\Rightarrow\; \mathtt{aaabbb\,aa} \to \varepsilon\,\mathtt{abbb}.$$

**Non-termination.** Swaps are bidirectional, so we can permute letters indefinitely; the ARS is not terminating.

**Equivalence classes & invariant.** Swaps preserve counts of letters. The rule $\mathtt{aa} \to \varepsilon$ preserves $\#a$ (mod 2). Thus the complete invariant is

$$\big(\#\mathtt{b},\ \#\mathtt{a} \bmod 2\big).$$

**Normal forms via terminating variant.** If we orient swaps one way (e.g. $\mathtt{ba} \to \mathtt{ab}$) and keep $\mathtt{aa} \to \varepsilon$, then every word terminates in a unique normal form:

$$\mathtt{b}^k \quad (\#a \text{ even}), \qquad \mathtt{a\,b}^k \quad (\#a \text{ odd}),$$

where $k = \#\mathtt{b}$.

**Exercise 5b — Analysis**

**Rules (variant):**

$$\mathtt{ab} \to \mathtt{ba}, \quad \mathtt{ba} \to \mathtt{ab}, \quad \mathtt{aa} \to \mathtt{a}.$$

**Non-termination.** Still non-terminating because of two-way swaps.

**Equivalence classes & invariant.** Swaps preserve #b. The rule aa→ a collapses any number of a's to exactly one (and in the equivalence closure we can also expand). Thus classes are determined by

$$\big(\#\mathtt{b},\ \mathbf{1}_{\{\#a\geq 1\}}\big),$$

i.e. the number of bs and whether there is at least one a.

**Normal forms via terminating variant.** If we orient ba→ab and keep aa→ a, $then the unique normal forms are$ : $\mathtt{b}^k \quad (\#a = 0), \qquad \mathtt{a}\,\mathtt{b}^k \quad (\#a \geq 1)$, with $k = \#\mathtt{b}$.

**Reflection & Question**

**Question.** In example 5b every word with at least one a collapses to the same class. Does this mean aaa and a are always equivalent in both directions or only in the equivalence closure?

## 2.4 Week 4

## Homework

Using a measure function (from the termination notes), prove termination of (1) Euclid's algorithm for $\gcd(a, b)$ and (2) a standard top-down merge sort.

**HW 4.1 — Euclid's Algorithm Terminates**

**Algorithm.**

```
while b != 0:
    temp = b
    b = a mod b
    a = temp
return a
```

**Assumptions.** $a, b \in \mathbb{N}$ and the loop guard is first checked with $b \geq 0$.

**Measure.** $\phi(a, b) = b \in \mathbb{N}$.

**Proof.** One loop step maps $(a, b)$ to $(b,\ a \bmod b)$. If $b \neq 0$, then $0 \leq a \bmod b < b$, hence

$$\phi(b,\ a \bmod b) = a \bmod b \ <\ b \ =\ \phi(a, b).$$

Thus $\phi$ strictly decreases at each taken iteration and always stays in $\mathbb{N}$. Since $>$ on $\mathbb{N}$ is well-founded, there is no infinite strictly decreasing chain; the loop terminates when $b = 0$.

**HW 4.2 — Merge Sort Terminates**

**Code (top-down).**

```
function merge_sort(arr, left, right):
    if left >= right: return
    mid = (left + right) // 2   // integer division
    merge_sort(arr, left, mid)
    merge_sort(arr, mid+1, right)
    merge(arr, left, mid, right)
```

**Measure.** For a call on interval $[left, right]$, let

$$\phi(left, right) \ =\ right - left + 1 \in \mathbb{N}.$$

**Proof.** If $left \geq right$, the function returns immediately. Otherwise $left < right$ and with integer division $left \leq mid < right$. The recursive calls are on $[left, mid]$ and $[mid{+}1, right]$ with measures

$$\phi(left, mid) = mid - left + 1 \;<\; right - left + 1 = \phi(left, right),$$

$$\phi(mid{+}1, right) = right - mid \;<\; right - left + 1 = \phi(left, right).$$

Hence each recursive call strictly decreases $\phi$. Because $\phi$ ranges over $\mathbb{N}$ and decreases along recursion, there is no infinite descent; all recursion bottoms out and the algorithm terminates. (No measure is needed for `merge`, which is non-recursive.)

**Reflection & Question**

**Question.** What's an example of a "bad" measure function that wouldn't actually prove termination?

## 2.5 Week 5

## Homework

Evaluate

$$(\lambda f.\, \lambda x.\, f(f(x)))\ (\lambda f.\, \lambda x.\, f(f(f\, x))).$$

**Solution**

Let $A \equiv \lambda f.\lambda x.\, f(f(x))$ and $B \equiv \lambda f.\lambda x.\, f(f(f\, x))$.

**Step 1.** Apply $A$ to $B$:

$$A\, B \;\to_\beta\; \lambda x.\, B\,(B\, x).$$

**Step 2.** $\alpha$-rename inside $B$ as $B \equiv \lambda f.\lambda y.\, f(f(f\, y))$ and compute:

$$B\, x \;=\; (\lambda f.\lambda y.\, f(f(f\, y)))\, x \;\to_\beta\; \lambda y.\, x(x(x\, y)).$$

Write $G \equiv \lambda y.\, x(x(x\, y))$. Then

$$\lambda x.\, B\,(B\, x) \;=\; \lambda x.\, B\, G \;\to_\beta\; \lambda x.\, \lambda y.\, G\big(G(G\, y)\big).$$

**Step 3.** Unroll $G$:

$$G\, y = x^3(y), \qquad G(G\, y) = x^6(y), \qquad G\big(G(G\, y)\big) = x^9(y).$$

**Answer**

$$\boxed{\lambda x.\, \lambda y.\, x(x(x(x(x(x(x(x(x\, y)))))))))} \;=\; \lambda x.\lambda y.\, x^9(y).$$

(Interpreting the terms as Church numerals, this is $3^2 = 9$.)

**Remark.** The left term applies a function twice, the right term thrice; composing them yields ninefold application.

## 2.6 Week 6

## Homework

Compute `fact 3` using the computation rules for `fix`, `let`, and `let rec`, and label each step with the rule used.

**Solution**

We use the standard sugars/rules:

$$\texttt{fix}\ F\ \to\ (F\,(\texttt{fix}\ F)) \qquad\qquad \texttt{let}\ x = e_1\ \text{in}\ e_2\ \to\ (\lambda x.\, e_2)\, e_1$$
$$\texttt{let rec}\ f = e_1\ \text{in}\ e_2\ \to\ \texttt{let}\ f = (\texttt{fix}\ (\lambda f.\, e_1))\ \text{in}\ e_2$$

and basic computation rules for conditionals and arithmetic:

$$0 = 0 \to \texttt{True}, \quad n{>}0 \Rightarrow (n = 0) \to \texttt{False}, \quad \texttt{if True then}\ A\ \texttt{else}\ B \to A, \quad \texttt{if False then}\ A\ \texttt{else}\ B \to B.$$

For brevity, write

$$F \stackrel{\text{def}}{=} \lambda f.\, \lambda n.\, \texttt{if}\ (n = 0)\ \texttt{then}\ 1\ \texttt{else}\ n * f(n-1).$$

**Derivation (each step labeled).**

$\texttt{let rec fact} = \lambda n.\, \texttt{if}\ (n = 0)\ \texttt{then}\ 1\ \texttt{else}\ n * \texttt{fact}\ (n-1)\ \ \texttt{in}\ \ \texttt{fact}\ 3$

$\xrightarrow{\text{def of } \texttt{let rec}} \texttt{let fact}\ = \texttt{fix}\ F\ \ \texttt{in}\ \ \texttt{fact}\ 3$

$\xrightarrow{\text{def of } \texttt{let}} (\lambda \texttt{fact}.\, \texttt{fact}\ 3)\ (\texttt{fix}\ F)$

$\xrightarrow{\beta} (\texttt{fix}\ F)\ 3$

$\xrightarrow{\text{def of } \texttt{fix}} (F\ (\texttt{fix}\ F))\ 3$

$\xrightarrow{\beta} (\lambda n.\, \texttt{if}\ (n = 0)\ \texttt{then}\ 1\ \texttt{else}\ n * (\texttt{fix}\ F)(n-1))\ 3$

$\xrightarrow{\beta} \texttt{if}\ (3 = 0)\ \texttt{then}\ 1\ \texttt{else}\ 3 * (\texttt{fix}\ F)(2)$

$\xrightarrow{\text{arith/if}} 3 * (\texttt{fix}\ F)(2)$

**Evaluate** $(\texttt{fix}\ F)(2)$ :

$(\texttt{fix}\ F)\ 2 \xrightarrow{\text{def of } \texttt{fix}} (F\ (\texttt{fix}\ F))\ 2 \xrightarrow{\beta} (\lambda n.\, \texttt{if}\ (n = 0)\ \texttt{then}\ 1\ \texttt{else}\ n * (\texttt{fix}\ F)(n-1))\ 2$

$\xrightarrow{\beta} \texttt{if}\ (2 = 0)\ \texttt{then}\ 1\ \texttt{else}\ 2 * (\texttt{fix}\ F)(1) \xrightarrow{\text{arith/if}} 2 * (\texttt{fix}\ F)(1)$

**Evaluate** $(\texttt{fix}\ F)(1)$ :

$(\texttt{fix}\ F)\ 1 \xrightarrow{\text{def of } \texttt{fix}} (F\ (\texttt{fix}\ F))\ 1 \xrightarrow{\beta} (\lambda n.\, \texttt{if}\ (n = 0)\ \texttt{then}\ 1\ \texttt{else}\ n * (\texttt{fix}\ F)(n-1))\ 1$

$\xrightarrow{\beta} \texttt{if}\ (1 = 0)\ \texttt{then}\ 1\ \texttt{else}\ 1 * (\texttt{fix}\ F)(0) \xrightarrow{\text{arith/if}} 1 * (\texttt{fix}\ F)(0)$

**Evaluate** $(\texttt{fix}\ F)(0)$ :

$(\texttt{fix}\ F)\ 0 \xrightarrow{\text{def of } \texttt{fix}} (F\ (\texttt{fix}\ F))\ 0 \xrightarrow{\beta} (\lambda n.\, \texttt{if}\ (n = 0)\ \texttt{then}\ 1\ \texttt{else}\ n * (\texttt{fix}\ F)(n-1))\ 0$

$\xrightarrow{\beta} \texttt{if}\ (0 = 0)\ \texttt{then}\ 1\ \texttt{else}\ 0 * (\texttt{fix}\ F)(-1) \xrightarrow{\text{arith/if}} 1$

**Back-substitute:** $1 * (\texttt{fix}\ F)(0) = 1 * 1 = 1, \quad 2 * (\texttt{fix}\ F)(1) = 2 * 1 = 2, \quad 3 * (\texttt{fix}\ F)(2) = 3 * 2 = 6.$

**Answer**

$$\boxed{\texttt{fact 3}\ \to\ 6}$$

**Reflection & Question**

**Question.** Could fix ever terminate incorrectly (or not at all) if the function has no base case?

## 2.7 Week 7

## Homework

**Given Grammar**

$$\begin{aligned}
&\text{Exp} \;\to\; \text{Exp} + \text{Exp1} \;\mid\; \text{Exp1}\\
&\text{Exp1} \;\to\; \text{Exp1} * \text{Exp2} \;\mid\; \text{Exp2}\\
&\text{Exp2} \;\to\; \text{Integer} \;\mid\; (\text{Exp}) \qquad (\text{with Integer} \;\to \texttt{n})
\end{aligned}$$

**Leftmost Derivations (quick checks)**

**2+1:** $\text{Exp} \Rightarrow \text{Exp} + \text{Exp1} \Rightarrow \text{Exp1} + \text{Exp1} \Rightarrow \text{Exp2} + \text{Exp1}$
$\Rightarrow \text{Integer} + \text{Exp1} \Rightarrow \texttt{2} + \text{Exp1} \Rightarrow \texttt{2} + \text{Exp2} \Rightarrow \texttt{2} + \text{Integer} \Rightarrow \texttt{2+1}.$

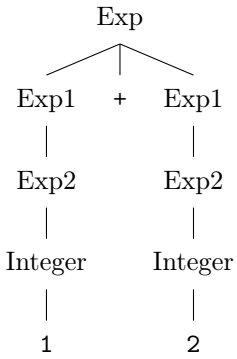
**1+2*3:** $\text{Exp} \Rightarrow \text{Exp} + \text{Exp1} \Rightarrow \text{Exp1} + \text{Exp1} \Rightarrow \text{Exp2} + \text{Exp1}$
$\Rightarrow \text{Integer} + (\text{Exp1} * \text{Exp2}) \Rightarrow \texttt{1} + (\text{Exp2} * \text{Exp2}) \Rightarrow \texttt{1} + (\text{Integer} * \text{Integer}) \Rightarrow \texttt{1+2*3}.$

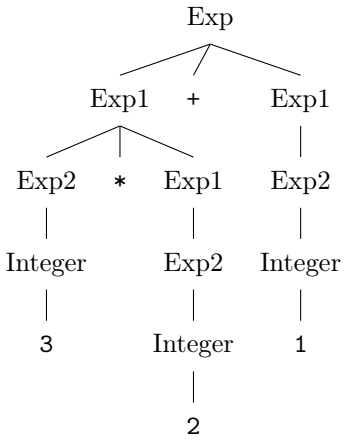
**1+(2*3):** $\text{Exp} \Rightarrow \text{Exp} + \text{Exp1} \Rightarrow \text{Exp1} + \text{Exp1} \Rightarrow \text{Exp2} + \text{Exp1} \Rightarrow \text{Integer} + \text{Exp1}$
$\Rightarrow \texttt{1} + \text{Exp2} \Rightarrow \texttt{1} + (\text{Exp}) \Rightarrow \texttt{1} + (\text{Exp1} * \text{Exp2}) \Rightarrow \texttt{1} + (\text{Integer} * \text{Integer}) \Rightarrow \texttt{1+(2*3)}.$

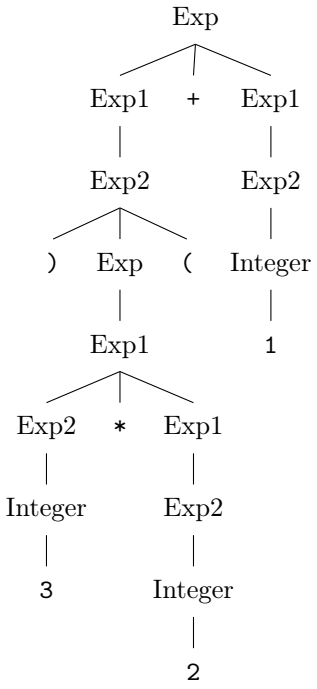
**(1+2)*3:** $\text{Exp} \Rightarrow \text{Exp1} \Rightarrow \text{Exp1} * \text{Exp2} \Rightarrow \text{Exp2} * \text{Exp2} \Rightarrow (\text{Exp}) * \text{Integer}$
$\Rightarrow (\text{Exp} + \text{Exp1}) * \texttt{3} \Rightarrow (\text{Exp1} + \text{Exp1}) * \texttt{3} \Rightarrow (\text{Integer} + \text{Integer}) * \texttt{3} \Rightarrow \texttt{(1+2)*3}.$

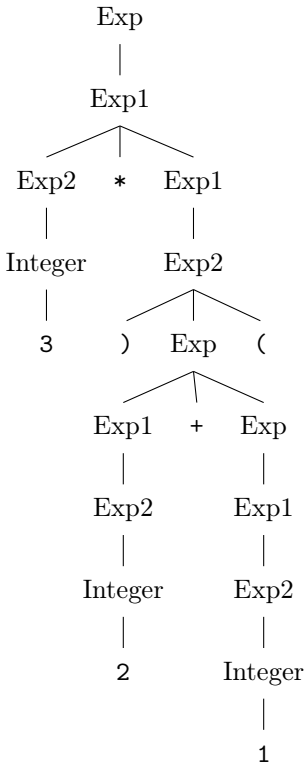
**1+2*3+4*5+6:** $\text{Exp} \Rightarrow \text{Exp} + \text{Exp1} \Rightarrow (\text{Exp} + \text{Exp1}) + \text{Exp1} \Rightarrow ((\text{Exp} + \text{Exp1}) + \text{Exp1}) + \text{Exp1}$
and expand each Exp1 with $\text{Exp1} \to \text{Exp1} * \text{Exp2}$ as needed.

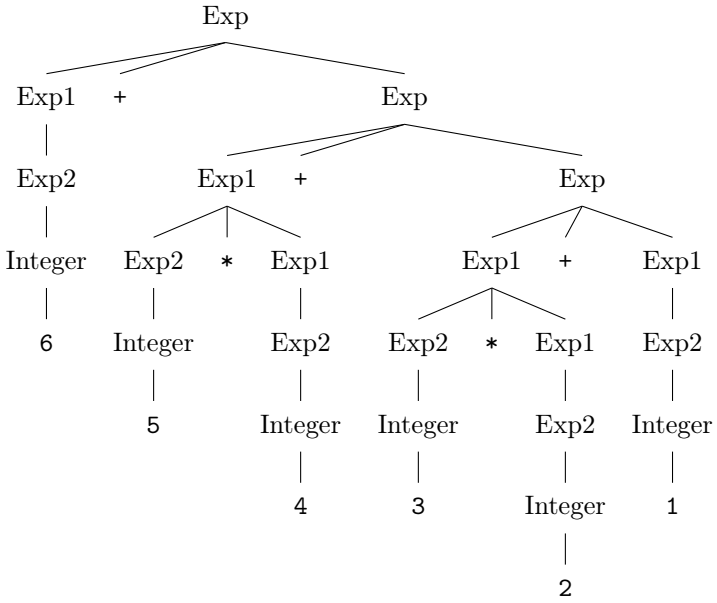
**Parse Trees**




Exp
Exp1
+
Exp1
Exp2
*
Exp1
Exp2
Integer
Exp2
Integer
3
Integer
1
2


Exp
Exp1
+
Exp1
Exp2
Exp2
)
Exp
(
Integer
Exp1
1
Exp2
*
Exp1
Integer
Exp2
3
Integer
2


Exp
Exp1
Exp2 * Exp1
Integer Exp2
3 ) Exp (
Exp1 + Exp
Exp2 Exp1
Integer Exp2
2 Integer
1


Exp
Exp1 + Exp
Exp2 Exp1 + Exp
Integer Exp2 * Exp1 Exp1 + Exp1
6 Integer Exp2 Exp2 * Exp1 Exp2
5 Integer Integer Exp2 Integer
4 3 Integer 1
2

## 2.8 Week 8

### Homework

Finished *Tutorial World* (NNG4). Below are my tactic scripts for Levels 5–8 and a matching English proof for Level 8.

**NNG Tutorial World — Solutions (Levels 5–8)**

```
-- Level 5: Adding zero
rw [add_zero b]
rw [add_zero c]
rfl

-- Level 6: Precision rewriting (same goal shape but focused rewrites)
rw [add_zero b]
rw [add_zero c]
rfl

-- Level 7: add_succ
rw [one_eq_succ_zero]
rw [add_succ]
rw [add_zero]
rfl

-- Level 8: 2 + 2 = 4
nth_rewrite 2 [two_eq_succ_one]
rw [add_succ 2]
nth_rewrite 1 [one_eq_succ_zero]
rw [add_succ 2]
rw [add_zero 2]
rw [   three_eq_succ_two]
rw [   four_eq_succ_three]
rfl
```

**Natural-language proof for Level 8 ($2 + 2 = 4$)**

We use Peano naturals where $\mathrm{succ}(n)$ is the successor, and

$$0, \quad 1 = \mathrm{succ}(0), \quad 2 = \mathrm{succ}(1), \quad 3 = \mathrm{succ}(2), \quad 4 = \mathrm{succ}(3).$$

Addition is defined by $a + 0 = a$ and $a + \mathrm{succ}(b) = \mathrm{succ}(a + b)$. Then:

$$\begin{aligned}
2 + 2 &= 2 + \mathrm{succ}(1) && (\text{since } 2 = \mathrm{succ}(1)) \\
&= \mathrm{succ}(2 + 1) && (\text{def. of addition}) \\
&= \mathrm{succ}\big(2 + \mathrm{succ}(0)\big) && (\text{since } 1 = \mathrm{succ}(0)) \\
&= \mathrm{succ}\big(\mathrm{succ}(2 + 0)\big) && (\text{def. of addition}) \\
&= \mathrm{succ}\big(\mathrm{succ}(2)\big) && (\text{since } 2 + 0 = 2) \\
&= \mathrm{succ}(3) = 4.
\end{aligned}$$

## 2.9 Week 9

### Homework

From *NNG4 Addition World*, complete Level 5 and provide two Lean solutions for

$$\texttt{add_right_comm}: \ \forall abc : \mathbb{N},\ a + b + c = a + c + b,$$

one *without induction* and one *with induction*. For each, include a corresponding pen-and-paper proof.

#### Level 5 — Statement

**Theorem 2.1** (Right-commutativity of addition)**.** *For all* $a, b, c \in \mathbb{N}$, $\quad a + b + c = a + c + b$.

#### Lean solution (without induction)

```
-- Using only associativity and commutativity of +
theorem add_right_comm (a b c : ) : a + b + c = a + c + b := by
  -- a + b + c = a + (b + c)
  rw [add_assoc]
  -- a + (b + c) = a + (c + b)
  rw [add_comm b c]
  -- a + (c + b) = a + c + b
  rw [add_assoc]
```

#### Pen-and-paper proof (corresponding to the non-inductive Lean script)

We use associativity and commutativity of $+$ on $\mathbb{N}$:

$$\begin{aligned} a + b + c &= a + (b + c) && \text{(associativity)} \\ &= a + (c + b) && \text{(commutativity)} \\ &= a + c + b && \text{(associativity).} \end{aligned}$$

Thus $a + b + c = a + c + b$.

#### Lean solution (with induction on $b$)

```
theorem add_right_comm (a b c : ) : a + b + c = a + c + b := by
  induction b with
  | zero =>
      -- goal: a + 0 + c = a + c + 0
      rw [add_zero, add_zero]
      rfl
  | succ d ih =>
      -- goal: a + succ d + c = a + c + succ d
      -- expose the IH by rewriting both sides
      rw [add_succ]      -- a + succ d + c = succ (a + d) + c
      rw [succ_add]      -- succ (a + d) + c = succ ((a + d) + c)
      rw [ih]            -- succ ((a + c) + d)
      rw [add_succ]      -- a + c + succ d = succ ((a + c) + d)
      rfl
```

**Pen-and-paper proof (corresponding to the inductive Lean script)**

**Claim.** For all $a, c \in \mathbb{N}$ and all $b \in \mathbb{N}$, we have $a + b + c = a + c + b$.

*Base case* $b = 0$. Using $x + 0 = x$,

$$a + 0 + c = a + c = a + c + 0.$$

*Inductive step.* Suppose $a + d + c = a + c + d$ for some $d$ (induction hypothesis). Using the successor laws

$$x + (y + 1) = (x + y) + 1 \quad \text{and} \quad (x + 1) + y = (x + y) + 1,$$

we compute

$$\begin{aligned} a + (d + 1) + c &= (a + d + 1) + c = (a + d + c) + 1 \\ &= (a + c + d) + 1 \qquad \text{(by IH)} \\ &= (a + c) + (d + 1) = a + c + (d + 1). \end{aligned}$$

Hence the property holds for $d + 1$, and by induction it holds for all $b \in \mathbb{N}$. □

**Reflection & Question**

**Question.** Could we use pattern matching on b instead of induction to do the same proof?

# 3 Synthesis

# 4 Evidence of Participation

# 5 Conclusion

# References